Atributos e Métodos Estáticos

Linguagem de Programação II

Universidade Estadual da Paraíba
Curso: Ciência da Computação

Sumário

1. O Problema: Informação que Pertence à Classe
2. O que é `static`
3. Atributos Estáticos
4. Métodos Estáticos
5. Estático vs. Instância
6. Boas Práticas
7. Juntando Tudo

Sumário

1. O Problema: Informação que Pertence à Classe
2. O que é `static`
3. Atributos Estáticos
4. Métodos Estáticos
5. Estático vs. Instância
6. Boas Práticas
7. Juntando Tudo

Você Já se Perguntou...

☺ Exemplo: Quantos alunos já foram criados?

```
1 Aluno a1 = new Aluno("Ana", 20);
2 Aluno a2 = new Aluno("Bruno", 22);
3 Aluno a3 = new Aluno("Carla", 19);
```

- Como saber que foram criados **3 alunos**?

Você Já se Perguntou...

☺ Exemplo: Quantos alunos já foram criados?

```
1 Aluno a1 = new Aluno("Ana", 20);
2 Aluno a2 = new Aluno("Bruno", 22);
3 Aluno a3 = new Aluno("Carla", 19);
```

- Como saber que foram criados **3 alunos**?
- Cada objeto guarda seus próprios dados (`nome`, `idade`...)

Você Já se Perguntou...

☺ Exemplo: Quantos alunos já foram criados?

```
1 Aluno a1 = new Aluno("Ana", 20);
2 Aluno a2 = new Aluno("Bruno", 22);
3 Aluno a3 = new Aluno("Carla", 19);
```

- Como saber que foram criados **3 alunos**?
- Cada objeto guarda seus próprios dados (`nome`, `idade`...)
- Mas a **contagem total** não pertence a nenhum objeto específico

Você Já se Perguntou...

☺ Exemplo: Quantos alunos já foram criados?

```
1 Aluno a1 = new Aluno("Ana", 20);
2 Aluno a2 = new Aluno("Bruno", 22);
3 Aluno a3 = new Aluno("Carla", 19);
```

- Como saber que foram criados **3 alunos**?
- Cada objeto guarda seus próprios dados (`nome`, `idade`...)
- Mas a **contagem total** não pertence a nenhum objeto específico
- Ela pertence à **classe como um todo**

A Tentativa Ingênua — e Por que Falha

Tentativa: guardar o contador em cada objeto

```java
public class Aluno {
    private String nome;
    private int contador; // cada objeto teria o seu!

    public Aluno(String nome) {
        this.nome = nome;
        this.contador++; // incrementa apenas o SEU contador
    }
}
```

☹ Erro Comum

`a1.contador = 1`, `a2.contador = 1`, `a3.contador = 1`

Cada objeto conta apenas a si mesmo — nunca chegamos ao total.

O Problema Visualizado

Cada objeto vive no seu mundo

Objeto	nome	contador (errado)
a1	Ana	1
a2	Bruno	1
a3	Carla	1

O que precisamos

Uma variável que seja **compartilhada por todos os objetos** da classe, mas que pertença à **classe em si** — não a nenhum objeto individualmente.

Sumário

1. O Problema: Informação que Pertence à Classe
2. O que é `static`
3. Atributos Estáticos
4. Métodos Estáticos
5. Estático vs. Instância
6. Boas Práticas
7. Juntando Tudo

A Palavra-chave `static`

☺ Definição: static

Um membro declarado com `static` pertence à **classe**, e não a qualquer instância (objeto). Existe **uma única cópia** na memória, independente de quantos objetos foram criados.

Membros de instância (sem static)

- Cada objeto tem a sua cópia
- `a1.nome`, `a2.nome` são independentes
- Acessados via objeto: `obj.membro`

Membros estáticos (com static)

- Existe uma única cópia para toda a classe
- Todos os objetos enxergam o mesmo valor
- Acessados via classe: `Classe.membro`

Analogia: Regras da Escola vs. Nota do Aluno

🤓 Analogia

- **Nota do aluno** → dado de **instância**: cada aluno tem a sua
- **Nome da escola** → dado **estático**: é o mesmo para todos os alunos
- **Regras do regulamento** → **estático**: aplicam-se a todos
- **Histórico individual** → **instância**: cada aluno tem o seu

Resumindo

Instância = meu dado $\times$ **Estático** = nosso dado

Sintaxe Básica

Declaração de membros estáticos

```
1  public class Aluno {
2
3      // Atributo estatico: pertence a Aluno (a classe)
4      private static int contador = 0;
5
6      // Atributo de instancia: pertence a cada objeto
7      private String nome;
8      private double nota;
9
10     // Metodo estatico
11     public static int getContador() {
12         return contador;
13     }
14 }
```

Sumário

1. O Problema: Informação que Pertence à Classe
2. O que é `static`
3. Atributos Estáticos
4. Métodos Estáticos
5. Estático vs. Instância
6. Boas Práticas
7. Juntando Tudo

Atributo Estático: Uma Cópia, Todos Compartilham

Declaração e inicialização

```java
public class Aluno {
    private static int contador = 0; // inicializado uma vez
    private String nome;

    public Aluno(String nome) {
        this.nome = nome;
        contador++; // incrementa a copia da classe
    }
}
```

No main:

```java
Aluno a1 = new Aluno("Ana");   // contador = 1
Aluno a2 = new Aluno("Bruno"); // contador = 2
Aluno a3 = new Aluno("Carla"); // contador = 3
```

Visualizando a Memória

Instâncias — cada uma na heap

a1 → { nome: “Ana” }

a2 → { nome: “Bruno” }

a3 → { nome: “Carla” }

Área estática — uma cópia da classe

`Aluno.contador` → **3**

(mesma cópia para a1, a2 e a3)

Ponto-chave

Quando a1, a2 e a3 leem `contador`, todos enxergam **o mesmo valor** — pois é uma única variável alocada na área estática da JVM.

Constantes Estáticas com `final`

😊 Definição: static final

Combinando `static` com `final` criamos **constantes** da classe: valores fixos que não mudam e são compartilhados por todos.

Exemplos

```
1 public class Aluno {
2     public static final double NOTA_MAX  = 10.0;
3     public static final double NOTA_MIN  = 0.0;
4     public static final int    CARGA_HORARIA = 60;
5 }
```

😄 Dica

Acesso ao Atributo Estático

Correto — via nome da classe

```java
// recomendado
int total = Aluno.getContador();

// constante
double max = Aluno.NOTA_MAX;
```

Funciona, mas evite — via objeto

```java
// compila, mas confunde:
// parece dado do objeto a1
int total = a1.getContador();
```

☹ Erro Comum

Acessar membro estático por um objeto (`a1.contador`) gera **aviso de compilador** no Java moderno. O Java aceita, mas deixa o código enganoso — parece dado de `a1`, mas é da classe inteira.

Sumário

1. O Problema: Informação que Pertence à Classe
2. O que é `static`
3. Atributos Estáticos
4. Métodos Estáticos
5. Estático vs. Instância
6. Boas Práticas
7. Juntando Tudo

O que é um Método Estático

☺ Definição: Método estático

Um método declarado com `static` pertence à **classe**. Pode ser chamado **sem criar um objeto** e **não tem acesso** ao `this` nem aos atributos de instância.

Sintaxe

```
1 public class Aluno {
2     private static int contador = 0;
3
4     // metodo estatico: acessa apenas 'contador' (estatico)
5     public static int getContador() {
6         return contador;
7     }
8 }
```

Você Já Usou Métodos Estáticos!

A classe Math é inteiramente estática

```java
double raiz  = Math.sqrt(16.0);   // Math.sqrt e estatico
double rand  = Math.random();     // sem new Math()
double maior = Math.max(7, 13);
double pi    = Math.PI;           // constante estatica
```

Outros exemplos do Java

```java
int n    = Integer.parseInt("42");      // Integer.parseInt
String s = String.valueOf(3.14);        // String.valueOf
int[]  a = {3,1,2};
java.util.Arrays.sort(a);              // Arrays.sort
```

😄 Dica

Regra de Ouro: o que o Estático Pode Acessar

Método estático PODE:

- Acessar atributos `static`
- Chamar outros métodos `static`
- Criar objetos (`new`)
- Receber objetos por parâmetro e acessar seus membros

Método estático NÃO PODE:

- Usar `this`
- Acessar atributos de instância diretamente
- Chamar métodos de instância diretamente

☹ Erro Comum

```
1 public static void exibir() {
2     System.out.println(this.nome); // ERRO: 'this' nao existe aqui!
3 }
```

Por que o `this` não Existe no Estático?

🤓 Analogia

Imagine a secretaria da escola ligando para o aluno pelo celular. Ela chama pelo **nome da escola** (a classe), não precisa de um aluno específico em frente a ela.

Se ela precisasse do **histórico individual** de um aluno, precisaria que ele estivesse presente (objeto concreto com `this`).

O método estático é a secretaria: age pela instituição, sem um "objeto específico na frente".

Motivo técnico

`this` aponta para o objeto atual. Num método estático, **não há objeto atual** — o método foi chamado direto na classe. Portanto, `this` não faz sentido e o Java proíbe seu uso.

Métodos Estáticos e Parâmetros de Objeto

Estático pode receber objetos por parâmetro

```
1  public class Aluno {
2      private String nome;
3      private double nota;
4
5      // estatico, mas recebe dois Alunos para comparar
6      public static Aluno melhorAluno(Aluno a, Aluno b) {
7          if (a.nota >= b.nota) return a;
8          return b;
9      }
10 }
```

Uso no main

```
1 Aluno a1 = new Aluno("Ana",    8.5);
2 Aluno a2 = new Aluno("Bruno", 7.0);
```

Sumário

1. O Problema: Informação que Pertence à Classe
2. O que é `static`
3. Atributos Estáticos
4. Métodos Estáticos
5. Estático vs. Instância
6. Boas Práticas
7. Juntando Tudo

Comparação Direta

Característica	**Instância**	**Estático (`static`)**
Pertence a	cada objeto	à classe
Cópias na memória	uma por objeto	uma única
Acessa `this`	sim	**não**
Acessa atrib. de instância	sim	**não** (direto)
Como chamar	`obj.metodo()`	`Classe.metodo()`
Precisa de objeto?	sim	**não**

Regra prática

Se o comportamento **depende de dados do objeto** → instância.
Se o comportamento **independe de qualquer objeto** → estático.

Exemplo Comparativo: `exibirDados` vs. `getContador`

`exibirDados()` — instância

```java
// Precisa saber QUAL aluno
public void exibirDados() {
    System.out.println(
      this.nome + " - " + this.nota
    );
}
// Uso:
a1.exibirDados();
```

`getContador()` — estático

```java
// Nao precisa de objeto
public static int getContador() {
    return contador;
}
// Uso:
Aluno.getContador();
```

♡ Checkpoint

Pergunta-teste: "O comportamento mudaria de acordo com o objeto?"
Sim → instância. Não → estático.

O Método `main` é Estático — Por quê?

Você já escreveu `static` sem perceber

```java
public static void main(String[] args) {
    Aluno a1 = new Aluno("Ana", 8.5);
    // ...
}
```

Explicação

- A JVM precisa executar `main` **sem criar nenhum objeto** da sua classe
- Ela chama `Main.main(args)` direto na classe
- Por isso `main` **obrigatoriamente** precisa ser `static`
- É o ponto de entrada — o código que cria os primeiros objetos

😄 Dica

Consequência: Tudo que `main` Usa Precisa ser `static`

O erro clássico com Scanner

```
1 public class Main {
2     Scanner sc = new Scanner(System.in); // atributo de instancia
3
4     public static void main(String[] args) {
5         String nome = sc.nextLine(); // ERRO: sc nao e static!
6     }
7 }
```

☹ Erro Comum

Erro: `Cannot make a static reference to the non-static field sc`
O `main` é estático — não existe objeto da classe `Main` sendo executado, portanto não existe `this.sc`.

A Solução: Declarar o `Scanner` como `static`

Correto — Scanner estático, acessível pelo main

```java
public class Main {
    // static: pertence a classe Main, nao a um objeto
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.print("Nome: ");
        String nome = sc.nextLine(); // funciona!

        System.out.print("Idade: ");
        int idade = sc.nextInt();

        Aluno a = new Aluno(nome, idade);
        a.exibirDados();
    }
}
```

Por que isso Acontece? A Regra Geral

Regra

Um método `static` só enxerga **diretamente** membros que também sejam `static`. Para acessar um membro de instância, ele precisa de uma **referência a um objeto** explícita.

Acesso direto — só funciona se static

`static` método acessa `static` campo: ✓

`static` método acessa campo de instância: ×

Via objeto — sempre funciona

```
Aluno a = new Aluno(...);
a.getNome() ✓
```

Mesmo dentro de um método `static`.

Analogia

A secretaria (método estático) não pode pegar o material de "um aluno genérico" — ela precisa ou de um material da escola (estático) ou de um aluno **específico** em mãos (objeto

Sumário

1. O Problema: Informação que Pertence à Classe
2. O que é `static`
3. Atributos Estáticos
4. Métodos Estáticos
5. Estático vs. Instância
6. Boas Práticas
7. Juntando Tudo

Quando Usar `static`

Use `static` quando...

- O dado **não varia de objeto para objeto** *(contador, constantes)*
- O método **não precisa de dados do objeto** *(fábricas, utilitários)*
- Você quer fornecer **utilitários de conveniência** sem exigir instanciação *(Math, Arrays)*
- Você cria **constantes** da classe *(NOTA_MAX, PI)*

Não use `static` quando...

- O dado muda de acordo com cada objeto (`nome`, `nota`)
- O método precisa de `this` para funcionar
- Você está tentando "simplificar" ignorando OO

Anti-padrão: a Classe Totalmente Estática

Código procedural disfarçado de OO

```java
public class Aluno {
    public static String nome;
    public static double nota;

    public static void exibir() {
        System.out.println(nome + " - " + nota);
    }
}

// main:
Aluno.nome = "Ana";
Aluno.nota = 8.5;
Aluno.exibir();
```

☹ Erro Comum

Classes Utilitárias: Uso Legítimo de `static`

Uma classe pode ser *toda* estática se for utilitária

```java
public class NotaUtils {
    public static final double NOTA_MAX = 10.0;
    public static final double NOTA_MIN = 0.0;

    // converte nota 0-10 em conceito A-F
    public static char notaParaConceito(double nota) {
        if (nota >= 9.0) return 'A';
        if (nota >= 7.0) return 'B';
        if (nota >= 5.0) return 'C';
        return 'F';
    }

    // construtor privado: nao instanciar
    private NotaUtils() {}
}
```

Combinando `static` com Encapsulamento

Atributo estático privado + getter estático público

```
1  public class Aluno {
2      // privado: ninguem acessa diretamente
3      private static int contador = 0;
4
5      public Aluno(String nome) {
6          this.nome = nome;
7          contador++;
8      }
9
10     // publico: leitura controlada
11     public static int getContador() {
12         return contador;
13     }
14     // sem setContador: nao faz sentido alterar externamente
15 }
```

Sumário

1. O Problema: Informação que Pertence à Classe
2. O que é `static`
3. Atributos Estáticos
4. Métodos Estáticos
5. Estático vs. Instância
6. Boas Práticas
7. Juntando Tudo

A Classe `Aluno` Completa

Combinando instância, estático e encapsulamento

```
1  public class Aluno {
2      // estatico: pertence a classe
3      private static int contador = 0;
4      public static final double NOTA_MAX = 10.0;
5      public static final double NOTA_MIN = 0.0;
6
7      // instancia: pertence a cada objeto
8      private String nome;
9      private double nota;
0
1      public Aluno(String nome, double nota) {
2          this.nome  = nome;
3          setNota(nota);    // validacao pelo setter
4          contador++;       // incrementa o contador da classe
5      }
```

A Classe `Aluno` Completa (cont.)

Getters, setter e método estático

```
1     // getter de instancia
2     public String getNome() { return nome; }
3     public double getNota() { return nota; }
4
5     // setter com validacao
6     public void setNota(double nota) {
7         if (nota < NOTA_MIN || nota > NOTA_MAX) return;
8         this.nota = nota;
9     }
0
1     // metodo estatico: retorna total de objetos criados
2     public static int getContador() { return contador; }
3
4     // metodo estatico utilitario
5     public static char notaParaConceito(double nota) {
6         if (nota >= 9.0) return 'A';
7         if (nota >= 7.0) return 'B';
```

Rastreando a Execução Passo a Passo

Código no main

```
1 Aluno a1 = new Aluno("Ana",   8.5);
2 Aluno a2 = new Aluno("Bruno", 7.0);
3 Aluno a3 = new Aluno("Carla", 4.0);
4 System.out.println(Aluno.getContador());
5 System.out.println(Aluno.notaParaConceito(8.5));
```

Linha	Ação	contador	a1.nota	a2.nota
1	new Aluno("Ana", 8.5)	1	8.5	—
2	new Aluno("Bruno", 7.0)	2	8.5	7.0
3	new Aluno("Carla", 4.0)	3	8.5	7.0
4	getContador() imprime	3	—	—
5	notaParaConceito(8.5) imprime 'B'	—	—	—

Saída do Programa

Console

```
3
B
```

O que cada linha prova

- **3**: `contador` foi incrementado a cada `new`, e é compartilhado — nenhum objeto o "zerou" sozinho
- **B**: `notaParaConceito` é estático, chamado sem objeto, e funcionou corretamente com nota 8.5

♡ Checkpoint

Você consegue dizer: qual membro é da **classe** e qual é do **objeto**?

Recapitulando

Atributo Estático

- `private static tipo nome`
- Uma cópia para toda a classe
- Persiste entre objetos
- Ex.: contadores, constantes

Método Estático

- `public static tipo metodo()`
- Chamado via `Classe.metodo()`
- Sem `this`, sem atributos de instância
- Ex.: utilitários, fábricas, `main`

Constante Estática

- `public static final tipo NOME`
- Valor fixo, imutável
- `MAIUSCULAS_COM_UNDERSCORE`
- Ex.: `NOTA_MAX`, `Math.PI`

Regra de decisão

- Depende do objeto? $\rightarrow$ instância
- Pertence à classe? $\rightarrow$ `static`

Próximos Passos

O que vem a seguir

- **Herança**: compartilhar comportamento entre classes usando `extends`
- **Polimorfismo**: objetos de tipos diferentes respondendo ao mesmo método de formas distintas
- **Interfaces**: contratos que uma classe deve cumprir
- **Coleções**: `ArrayList`, `HashMap` — manipulando grupos de objetos

😊 Dica

Antes de avançar: pratique criando uma classe com atributo estático contador, constante `static final`, e ao menos um método estático utilitário. Compilar e executar é a melhor forma de fixar.